

MANUAL DE ARQUITECTURA Y MODIFICACIONES

Sistema Web de Acceso Unico a Videos con Control PostgreSQL

1. Funcionamiento de las 3 Rutas Independientes

La aplicacion web utiliza un unico servidor en Flask que gestiona 3 rutas URL independientes:

- * <https://tu-web.onrender.com/positivo> -> Acceso al video Positivo
- * <https://tu-web.onrender.com/neutro> -> Acceso al video Neutro
- * <https://tu-web.onrender.com/negativo> -> Acceso al video Negativo

Flujo de Acceso del Usuario:

1. Al ingresar a cualquiera de los enlaces anteriores, el usuario ve el Formulario de Ingreso (Nombre y Apellido).
2. Al presionar 'INGRESAR', el servidor consulta la base de datos PostgreSQL filtrando por Nombre + Apellido + Tipo de Video.
3. Si NO existe registro previo para ese video especifico, guarda el acceso en PostgreSQL y redirige al reproductor del video de 5 minutos.
4. Si YA existe el registro, el servidor bloquea el acceso mostrando la pantalla 'bloqueado.html'.
5. Seguridad de URL directas: Si un usuario intenta entrar directamente a '/ver/negativo' sin haber completado el formulario, el servidor detecta que no tiene sesion validada y lo redirige automaticamente al formulario.

2. Estructura de la Base de Datos (PostgreSQL)

Se utiliza una unica tabla 'personas' que incluye la columna 'tipo_video' para diferenciar el acceso de cada participante:

```
CREATE TABLE IF NOT EXISTS personas (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    apellido VARCHAR(100) NOT NULL,  
    tipo_video VARCHAR(50) NOT NULL,  
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3.Codigo del Servidor Principal (app.py)

```
import os  
from flask import Flask, render_template, request, redirect, url_for, flash, session  
import pg8000.native  
from dotenv import load_dotenv  
  
load_dotenv()  
app = Flask(__name__)  
app.secret_key = os.getenv("SECRET_KEY", "secreto_desarrollo_123")  
  
def get_db_connection():  
    return pg8000.native.Connection(  
        host=os.getenv("DB_HOST", "localhost"),  
        database=os.getenv("DB_NAME", "proyecto_videos"),  
        user=os.getenv("DB_USER", "postgres"),  
        password=os.getenv("DB_PASSWORD", ""),  
        port=int(os.getenv("DB_PORT", 5432))  
    )  
  
# DICCIONARIO DE VIDEOS (Aqui se modifican los enlaces de los videos)  
VIDEOS = {  
    "positivo": {  
        "titulo": "Evaluacion - Video Positivo",  
        "url": "https://www.w3schools.com/html/mov_bbb.mp4",  
        "badge": "Fase Positiva",  
        "clase_badge": "badge-bueno"  
    },  
    "neutro": {  
        "titulo": "Evaluacion - Video Neutro",
```

```

        "url": "https://www.w3schools.com/html/movie.mp4",
        "badge": "Fase Neutra",
        "clase_badge": "badge-neutro"
    },
    "negativo": {
        "titulo": "Evaluacion - Video Negativo",
        "url": "https://www.w3schools.com/html/mov_bbb.mp4",
        "badge": "Fase Negativa",
        "clase_badge": "badge-malo"
    }
}

@app.route("/<tipo_video>", methods=["GET", "POST"])
def login_video(tipo_video):
    tipo = tipo_video.lower()
    if tipo not in VIDEOS:
        return "Pagina no encontrada", 404
    if request.method == "POST":
        nombre = request.form.get("nombre", "").strip()
        apellido = request.form.get("apellido", "").strip()
        if not nombre or not apellido:
            flash("Por favor, completa ambos campos.", "danger")
            return redirect(url_for("login_video", tipo_video=tipo))
        try:
            conn = get_db_connection()
            existente = conn.run(
                "SELECT id FROM personas WHERE LOWER(nombre) = LOWER(:n) AND LOWER(apellido) = LOWER(:a) AND tipo_video
= :t",
                n=nombre, a=apellido, t=tipo
            )
            if existente:
                conn.close()
                return render_template("bloqueado.html")
            conn.run(
                "INSERT INTO personas (nombre, apellido, tipo_video) VALUES (:n, :a, :t)",
                n=nombre, a=apellido, t=tipo
            )
            conn.close()
            session["usuario_validado"] = f"{nombre}_{apellido}_{tipo}"
            return redirect(url_for("ver_video_unico", tipo_video=tipo))
        except Exception as e:
            print("Error DB:", e)
            flash("Error de conexion a la base de datos.", "danger")
            return redirect(url_for("login_video", tipo_video=tipo))
    return render_template("index.html", tipo=tipo, info=VIDEOS[tipo])

@app.route("/ver/<tipo_video>")
def ver_video_unico(tipo_video):
    tipo = tipo_video.lower()
    if not session.get("usuario_validado") or tipo not in VIDEOS:
        return redirect(url_for("login_video", tipo_video=tipo if tipo in VIDEOS else "positivo"))
    return render_template("videos.html", video_info=VIDEOS[tipo])

if __name__ == "__main__":
    app.run(debug=True)

```

4. Manual de Mantenimiento: Como Cambiar los Videos y Ajustes

PASO 1: Subir tus videos editados de 5 minutos

- * Sube tus archivos de video (.mp4) a Supabase Storage, Cloudflare R2 o tu servidor de almacenamiento.
- * Copia el enlace directo publico de cada video (ejemplo: https://mi-servidor.com/video_positivo.mp4).

PASO 2: Reemplazar las URLs en app.py

1. Abre el archivo 'app.py' en Geany en tu computadora.
2. Ubica el diccionario 'VIDEOS' al principio del archivo.
3. Reemplaza las URLs temporales por tus enlaces reales:
"positivo": { "url": "TU_ENLACE_POSITIVO_AQUI.mp4", ... }
"neutro": { "url": "TU_ENLACE_NEUTRO_AQUI.mp4", ... }
"negativo": { "url": "TU_ENLACE_NEGATIVO_AQUI.mp4", ... }
4. Guarda el archivo con Ctrl + S.

PASO 3: Subir los cambios a GitHub

1. Entra a tu repositorio en GitHub (<https://github.com/locololo/Proyecto-videos->).
2. Haz clic en 'app.py', selecciona el icono del lapiz (Editar) o sube el archivo actualizado.
3. Presiona el boton verde 'Commit changes'.

PASO 4: Despliegue automatico en Render

- * Render esta vinculado a tu cuenta de GitHub, por lo que cada vez que hagas un 'Commit' en GitHub, Render detectara el cambio y actualizara tu sitio web en internet automaticamente en 2 o 3 minutos.

5. Resumen del Frontend Simplificado (Unico Reproductor)

* **templates/videos.html:** Contiene un solo reproductor <video> que recibe de Flask la URL y el titulo.

Incluye un boton gigante de 'START VIDEO' necesario para otorgar permisos de audio en el navegador.

* **static/js/main.js:** Escucha el evento nativo de 'pause' para forzar el re-inicio del video de inmediato,
previene adelantos/retrocesos comparando 'currentTime', bloquea el clic derecho y muestra el cartel final
al concluir los 5 minutos de reproduccion.

* **static/css/style.css:** Mantiene la proporcion cine 16:9 con un ancho HD de 1000px sobre fondo oscuro.